1. A long-running agent conversation will eventually exceed the model's context window. Which Anthropic capability automatically condenses earlier conversation server-side to keep the session going?

A. Stop sequences
B. Adding more tools
C. Compaction (server-side summarization of earlier turns)
D. Lowering temperature

Correct:
Compaction (server-side summarization of earlier turns)
Explanation:
Compaction provides server-side summarization that automatically condenses earlier parts of a conversation, enabling long-running sessions beyond context limits with minimal integration work. It is offered for recent models. This is a key reliability tool for agents that run for a long time.

2. A team relies on assistant message prefilling to steer the response, but they also enabled JSON outputs mode and now get a 400 error. What is the correct understanding?

A. Prefilling requires structured outputs to function
B. The error is unrelated to JSON outputs mode
C. Prefilling always works with JSON outputs mode
D. JSON outputs mode is incompatible with message prefilling; you must choose one approach for that request

Correct:
JSON outputs mode is incompatible with message prefilling; you must choose one approach for that request
Explanation:
Structured outputs in JSON outputs mode do not work with message prefilling (it returns an error), so you cannot combine them on the same request. If prefilling is essential, you forgo JSON outputs mode for that call (strict tool use may still apply). Knowing these compatibility boundaries is part of designing reliable output contracts.

3. An architect must choose a model for a high-volume, latency-sensitive classification step inside a larger agent, where most decisions are simple. Which model selection strategy aligns with Anthropic's guidance?

A. Always use the largest Opus model everywhere for maximum quality
B. Use a faster, cheaper model such as Haiku for the simple high-volume step, reserving larger models for genuinely hard reasoning steps
C. Route every step to a random model to balance load
D. Use a fine-tuned model because base models cannot classify

Correct:
Use a faster, cheaper model such as Haiku for the simple high-volume step, reserving larger models for genuinely hard reasoning steps
Explanation:
Anthropic's model lineup is tiered: Haiku for fast, low-cost, high-volume tasks; Sonnet for balanced general work; Opus for the hardest reasoning. A good architecture matches each step to the cheapest model that meets the quality bar, using Haiku for simple high-volume classification and reserving Opus for complex reasoning. This optimizes both cost and latency.

4. A security team wants to grant a Claude Code subagent access to only Read, Grep, and Glob and nothing else. What is the recommended way to enforce this least-privilege boundary?

A. Configure per-subagent tool access (allowed tools), since subagents support granting specific tools for security
B. Delete all other tools from the user's machine
C. Trust the subagent to ignore other tools
D. Set max_tokens to a small number

Correct:
Configure per-subagent tool access (allowed tools), since subagents support granting specific tools for security
Explanation:
Subagents support tool control: you can grant a specific set of tools per subagent, enforcing least privilege. Restricting a code-review subagent to Read, Grep, and Glob prevents it from editing or executing anything. This is enforced configuration, not a behavioural hope.

5. A team wants Claude to reason before answering a hard problem, producing real reasoning (thinking) blocks rather than a hidden trick. On current models, which capability and parameter control this?

A. Adaptive thinking (thinking: type 'adaptive'), with reasoning depth tuned by the effort parameter
B. output_format generating a reasoning field
C. stop_sequences controlling reasoning length
D. tool_choice forcing a thinking tool

Correct:
Adaptive thinking (thinking: type 'adaptive'), with reasoning depth tuned by the effort parameter
Explanation:
On current models you use adaptive thinking (thinking: type 'adaptive'), and Claude decides whether and how much to think per request. Thinking is already on for models such as Claude Opus 5.5 and Sonnet 5.5, while Claude Opus 4.8 and Sonnet 4.6 turn it on with type 'adaptive'. You steer depth and token spend with the effort parameter in output_config (low up to max; the available levels vary by model). The fixed budget_tokens control is deprecated on the Claude 4.6 models and rejected with a 400 error on Claude 4.7 and later models.

6. A developer wants to use the Agent SDK for a simple one-shot task: give Claude a job and get a result back, with no persistent session. Which interface is most appropriate?

A. The query() function, which runs a one-shot task and yields messages as Claude works
B. ClaudeSDKClient for a long-lived persistent session
C. A raw socket connection
D. The web chat Ul embedded in code

Correct:
The query() function, which runs a one-shot task and yields messages as Claude works
Explanation:
The query() function is designed for one-shot tasks: you give Claude a job and receive results as an async stream of messages as it reasons. ClaudeSDKClient is for persistent sessions, custom tools, hooks, and long-running interactions. Choosing query() keeps simple tasks simple.

7. An MCP server author wants Claude users to invoke a curated, reusable interaction template (for example 'summarize this PR in our house style'). Which MCP primitive should they implement?

A. A prompt
B. A resource
C. A stop sequence
D. A transport

Correct:
A prompt
Explanation:
MCP prompts are reusable templates that package ready-to-use conversation context for a model, which is exactly what a curated 'house style' interaction is. Resources expose data, and tools perform actions. Prompts let server authors ship best-practice interaction patterns that clients can surface to users.

8. An orchestrator agent delegates subtasks to workers, but workers sometimes need slightly different instructions depending on the subtask. What is the recommended way for the orchestrator to convey this in an orchestrator-worker design?

A. Send no instructions and let workers guess
B. Hard-code one identical prompt for every worker regardless of subtask
C. Replace workers with a single sequential chain
D. Have the orchestrator dynamically generate task-specific instructions for each worker based on its decomposition

Correct:
Have the orchestrator dynamically generate task-specific instructions for each worker based on its decomposition
Explanation:
In orchestrator-worker, the central model decomposes the problem and dynamically generates the specific subtask and instructions for each worker. Because the subtasks are not known in advance, the orchestrator tailors each delegation rather than reusing one fixed prompt. The orchestrator then synthesizes worker outputs into a final result.

9. A long agentic task using interleaved thinking accumulates many thinking blocks, bloating context. Which context-editing strategy specifically targets this?

A. Thinking block clearing, which removes earlier thinking content no longer needed
B. Adding more stop sequences
C. Forcing strict tool use
D. Raising temperature

Correct:
Thinking block clearing, which removes earlier thinking content no longer needed
Explanation:
Context editing includes thinking block clearing, which removes earlier reasoning blocks that are no longer needed to free context in long agentic workflows. It complements tool result clearing. These deliberate edits keep long runs within the context budget.

10. An architect wants Claude to use a company's GitHub, an internal database, and a browser automation service, all through one consistent interface in Claude Code. What is the recommended integration approach?

A. Hard-code each integration into the prompt as text
B. Connect each system via its MCP server so each exposes tools/resources over the standard protocol while Claude reasons about usage
C. Train a custom model per system
D. Use stops sequences to call external APIs

Correct:
Connect each system via its MCP server so each exposes tools/resources over the standard protocol while Claude reasons about usage
Explanation:
Connecting each external system through its MCP server exposes tools, resources, and prompts over a consistent protocol, letting Claude Code reason about what to do while the servers own connectivity. This avoids bespoke per-system glue and keeps integrations reusable. It is the canonical pattern for wiring Claude to external systems.

11. Anthropic recommends structuring complex prompts with clear sections. Which approach aligns with this guidance for a multi-part instruction?

A. Use clearly delimited sections (for example XML-style tags or headers) to separate instructions, context, and examples
B. Write one unbroken paragraph mixing instructions and data
C. Hide the instructions inside the user's pasted data
D. Randomize the order of sections each call

Correct:
Use clearly delimited sections (for example XML-style tags or headers) to separate instructions, context, and examples
Explanation:
Anthropic's prompting guidance favours clear structure, such as delimited sections or XML-style tags, to separate instructions, context, and examples so the model parses each part correctly. This is especially important when mixing trusted instructions with untrusted or large input. Clear delimiting improves both accuracy and injection resistance.

12. An architect must decide between RAG and relying on a 1M-token context window for a knowledge base of about 300K tokens that fits comfortably. According to current guidance, which framing is correct?

A. Long context makes accuracy impossible
B. RAG is always mandatory regardless of size
C. With large context windows, RAG is a deliberate choice rather than a default; for data that fits, loading context directly can be simpler, while RAG suits data exceeding the window or where retrieval is more economical at scale
D. RAG and long context cannot coexist

Correct:
With large context windows, RAG is a deliberate choice rather than a default; for data that fits, loading context directly can be simpler, while RAG suits data exceeding the window or where retrieval is more economical at scale
Explanation:
With reliable large context windows, RAG becomes a deliberate engineering choice rather than a default requirement: when data fits in context, loading it directly can be simpler and effective, while RAG remains valuable when data exceeds the window or retrieval is more economical at scale. Architects weigh size, cost, and freshness. Neither approach is universally mandatory.

13. An architect notices that with thinking enabled on a recent model, the billed output token count exceeds the visible response (and summarized thinking) tokens. What is the correct explanation?

A. Extended thinking is free, so the discrepancy is random
B. Output tokens are always double-counted
C. It is a billing bug to dispute
D. You are billed for the full thinking tokens the model generates, while the raw reasoning is never returned (you see only a summary, or nothing if display is omitted) so billed output exceeds the visible tokens

Correct:
You are billed for the full thinking tokens the model generates, while the raw reasoning is never returned (you see only a summary, or nothing if display is omitted) so billed output exceeds the visible tokens
Explanation:
The raw chain of thought is not returned: depending on the display setting you see a summary or nothing, but you are charged for the full thinking tokens the model actually generated. That is why billed output can exceed the visible response token count. Understanding this prevents cost surprises when enabling thinking.

14. An architect wants to enforce a complex nested JSON schema for a financial report. Why are structured outputs preferred over manually validating and re-prompting on failure?

A. Structured outputs constrain generation to the grammar so valid output is produced directly, avoiding retry loops and parse failures
B. Manual re-prompting guarantees schema conformance on the first try
C. Manual re-prompting is always cheaper and faster
D. Structured outputs ignore the schema entirely

Correct:
Structured outputs constrain generation to the grammar so valid output is produced directly, avoiding retry loops and parse failures
Explanation:
Structured outputs constrain token generation to a grammar compiled from the schema, so the model emits conformant JSON directly rather than relying on validate-and-retry loops that add latency, cost, and failure modes. This is more reliable for complex nested schemas. Retry loops can still fail repeatedly and are harder to operate.

15. An architect wants to expose read-only reference data (a product catalogue) to Claude through MCP so the model can pull it as context, without it being an action with side effects. Which primitive fits best?

A. A tool, because all data must be a tool
B. A prompt template
C. A resource, which exposes data at a URI for the client to read as context
D. A transport

Correct:
A resource, which exposes data at a URI for the client to read as context
Explanation:
Resources expose data at a URI for the client to read as contextual information, which fits read-only reference data like a catalogue. Tools are for executable actions that compute or have effects, and prompts are reusable conversation templates. Matching read-only data to a resource keeps the action surface clean and intentful.

16. An architect wants Claude to stop generating as soon as it produces a specific delimiter token like '</answer>'. Which API parameter is designed for this?

A. output_config.format
B. effort
C. tool_choice
D. stop_sequences

Correct:
stop_sequences
Explanation:
The stop_sequences parameter tells the API to stop generating when one of the specified strings appears, which is useful for bounding output at a known delimiter. tool_choice controls tool selection, the effort parameter governs thinking depth and token spend, and output_config.format enforces a JSON schema. Picking the parameter that matches the intent is basic API fluency.

17. An architect wants automated behaviour such as 'whenever Claude finishes, run the test suite.' Which Claude Code mechanism actually executes this automatically (rather than depending on the model remembering)?

A. A note in CLAUDE.md asking Claude to remember
B. Hoping the model recalls the instruction each time
C. A Stop hook configured in settings that runs the test command when the agent stops
D. A glossary entry

Correct:
A Stop hook configured in settings that runs the test command when the agent stops
Explanation:
Automated 'whenever X happens, do Y' behaviours are implemented with hooks, because the harness (not the model) executes them deterministically at the chosen lifecycle event. A Stop hook runs your command when the agent stops. Relying on the model to remember a CLAUDE.md note is best-effort and not guaranteed.

18. An architect connects Claude Code to a company database via MCP so Claude can query it while reasoning about a task. In this setup, what is the correct division of labour?

A. The MCP server decides the task plan and Claude just relay it
B. The MCP server handles the integration and exposes tools/resources; Claude reasons about what to do with them
C. Claude implements the database driver itself in the prompt
D. The MCP server replaces the need for any model

Correct:
The MCP server handles the integration and exposes tools/resources; Claude reasons about what to do with them
Explanation:
MCP servers handle the integration with external systems and expose capabilities (tools, resources, prompts) over a standard protocol, while Claude reasons about when and how to use them. This separation lets the model focus on planning while the server owns connectivity. It also makes integrations reusable across clients.

19. An agent's input may contain untrusted user text that tries to override its instructions (prompt injection). Which architectural guardrail most directly reduces this risk?

A. Concatenate user text directly into the system prompt to save tokens
B. Give the agent unrestricted tool access so it can defend itself
C. Disable the system prompt entirely
D. Place untrusted content in clearly delimited input sections, keep authoritative instructions in the system prompt, and validate/limit tool actions

Correct:
Place untrusted content in clearly delimited input sections, keep authoritative instructions in the system prompt, and validate/limit tool actions
Explanation:
Defending against prompt injection combines clear separation of untrusted input from trusted instructions, keeping authoritative directives in the system prompt, and constraining what tools can do so a successful injection has limited blast radius. Delimiting and labelling untrusted content help the model treat it as data, not instructions. Layered guardrails matter more than any single trick.

20. An architect designs a tool for Claude. Which element most influences whether Claude calls the tool correctly and at the right time?

A. Whether the tool name contains numbers
B. A clear, descriptive tool description plus a well-specified input_schema (JSON Schema) for parameters
C. The order tools were registered alphabetically
D. The colour of the tool's icon

Correct:
A clear, descriptive tool description plus a well-specified input_schema (JSON Schema) for parameters
Explanation:
Claude decides when and how to call a tool primarily from its description and its input_schema, which define the tool's purpose and the shape of its parameters. A clear description and precise schema dramatically improve correct, well-timed tool calls. Anthropic treats tool descriptions as prompt engineering for the tool interface.

21. A reliability review asks how to detect regressions before they reach users when prompts, tools, or models change. What is the most robust practice?

A. Ship changes directly to production and watch for complaints
B. Maintain an automated eval suite of representative cases and run it on every change, gating releases on results
C. Rely on a single manual spot-check
D. Only test after an incident occurs

Correct:
Maintain an automated eval suite of representative cases and run it on every change, gating releases on results
Explanation:
An automated evaluation suite of representative cases, run on every change and used to gate releases, catches regressions before users do. Anthropic emphasizes building on measurement and adding complexity only when evals justify it. Spot-checks and post-incident testing are far weaker safety nets.

22. A team wants Claude to call several independent tools at once in a single turn to reduce round trips. Which mechanism enables this, and how can it be disabled when ordering matters?

A. Setting max_tokens to 1 enables parallel calls
B. You must run a separate model instance per tool
C. Parallel tool calls are supported by default; set disable_parallel_tool_use to true to force one tool call at a time
D. Parallel tool calls are impossible in the Messages API

Correct:
Parallel tool calls are supported by default; set disable_parallel_tool_use to true to force one tool call at a time
Explanation:
Claude can emit multiple tool_use blocks in one turn (parallel tool use) when the calls are independent, reducing round trips. If you need strictly sequential calls, you set disable_parallel_tool_use to true so Claude makes one tool call at a time. Choosing this based on dependency structure is a tool-design decision.

23. A high-stakes financial agent must never act on a hallucinated tool result. Beyond returning errors honestly, which additional architectural guardrail most improves trustworthiness of its actions?

A. Increase max_tokens so the agent writes longer justifications
B. Verify critical tool outputs with an independent check or human review before the agent acts on them
C. Let the agent self-certify its own outputs with no external check
D. Remove all logging to reduce overhead

Correct:
Verify critical tool outputs with an independent check or human review before the agent acts on them
Explanation:
For high-stakes actions, verifying critical results with an independent check (a second tool, a deterministic validator, or human review) before acting reduces the chance of acting on a wrong or hallucinated value. Self-certification alone is insufficient because the same model that erred also approves it. Layered verification is a core reliability guardrail.

24. Anthropic's prompting guidance for capable models stresses telling the model what TO do. Which prompt phrasing best follows this?

A. 'Respond with a single valid JSON object containing the keys name and total, and nothing else.
B. 'Do not do anything wrong.
C. 'Maybe give some JSON if you feel like it.
D. Don't mess it up.

Correct:
'Respond with a single valid JSON object containing the keys name and total, and nothing else.
Explanation:
Effective prompts are explicit and positive about the desired behaviour and output, stating exactly what to produce (a single valid JSON object with specific keys and nothing else). Vague or purely negative instructions leave too much room for interpretation. Specificity about format and content is a core best practice.

25. An architect wants the most reliable way to get clean JSON for a downstream parser while preserving the ability to use citations on the same request. Given the compatibility constraints, what should they do?

A. Disable the model
B. Use stops sequences to force both
C. Recognize that JSON outputs mode does not work with citations, and choose one: either citations OR JSON outputs mode for that request
D. Use JSON outputs mode together with citations, since they are fully compatible

Correct:
Recognize that JSON outputs mode does not work with citations, and choose one: either citations OR JSON outputs mode for that request
Explanation:
JSON outputs mode does not work with citations (it returns an error), so on a single request you must choose between them. If citations are required, you forgo JSON outputs mode and validate JSON another way; if guaranteed JSON is required, you drop citations. Knowing these incompatibilities prevents 400 errors in production.

26. A subagent in the Agent SDK keeps prompting for permission on tools the parent already approved. What is the correct explanation and fix?

A. Each subagent may request permission separately for its own tool calls; auto-approve specific tools with PreToolUse hook or configure permission rules, which subagents inherit from the parent
B. A subagent reuses every approval the parent has given, so the prompts indicate a bug to ignore
C. Lower the model temperature to stop the prompts
D. Permissions cannot be configured for subagents at all

Correct:
Each subagent may request permission separately for its own tool calls; auto-approve specific tools with PreToolUse hooks or configure permission rules, which subagents inherit from the parent
Explanation:
When you spawn subagents, each one may request permission separately for its own tool calls, so approvals given interactively in the parent don't necessarily cover them. Anthropic's Agent SDK docs recommend PreToolUse hooks that auto-approve specific tools, or permission rules, which subagents inherit from the parent conversation. This removes repeated prompts while keeping the boundary explicit.

27. A team wants Claude Code to consistently follow their repository's conventions, build commands, and architecture rules in every session without re-explaining them. Which mechanism is designed for this?

A. A .gitignore entry listing the rules
B. Renaming files so Claude infers conventions
C. A CLAUDE.md file in the repository that Claude reads each session as project context
D. A one-time chat message that is forgotten after the session

Correct:
A CLAUDE.md file in the repository that Claude reads each session as project context
Explanation:
CLAUDE.md is the project's source of truth that Claude Code reads at the start of each session, anchoring behaviour to your conventions, commands, and architecture without re-explaining them. It persists across sessions in the repo. This is the canonical place for durable project rules.

28. A long-running autonomous agent must handle tasks that span many tool calls and thinking steps. Which model capability lets Claude reason between tool calls and chain multiple tool calls with reasoning steps in between?

A. Prompt caching
B. Batch processing
C. Stop sequences
D. Interleaved thinking (automatic on models with adaptive thinking), where Claude can think between tool calls

Correct:
Interleaved thinking (automatic on models with adaptive thinking), where Claude can think between tool calls
Explanation:
Interleaved thinking lets Claude reason between tool calls, considering a tool's result before deciding the next action and chaining multiple tool calls with reasoning steps in between. On models with adaptive thinking, such as Claude Opus 5.5, Sonnet 5.5 and Sonnet 4.6, it is automatic with no beta header; Claude Opus 4.5 and earlier Claude 4 models need the interleaved-thinking beta header, and Claude Haiku 4.5 does not support it. This suits agentic loops where each step depends on prior results.

29. When Claude decides to use a client tool, what does the API response contain and what does the application do next?

A. A plain text answer with no structured call
B. An error, because client tools are not supported
C. stop_reason 'tool_use' with one or more tool_use blocks; the app executes them and returns tool_result blocks referencing the matching tool_use_id
D. stop_reason 'end_turn' and the tool runs automatically on Anthropic's servers

Correct:
stop_reason 'tool_use' with one or more tool_use blocks; the app executes them and returns tool_result blocks referencing the matching tool_use_id
Explanation:
For client tools, Claude returns stop_reason 'tool_use' with tool_use content blocks (name, input, and an id). Your application executes the operation and sends back a tool_result block referencing the matching tool_use_id. This request-execute-return cycle is the tool-use loop for client tools.

30. A team wants reusable, invokable commands like /deploy-check that bundle a multi-step prompt for Claude Code. What is the modern way to define these, and how are skills different?

A. Skills run on a separate server and cannot use tools
B. Slash commands can only be built-in and cannot be customized
C. Define a skill in .claude/skills/ (older.claude/commands/ files still work); either one creates /name, and a skill can also be loaded by Claude automatically when relevant
D. Slash commands are deprecated and have no replacement

Correct:
Define a skill in .claude/skills/ (older.claude/commands/ files still work); either one creates /name, and a skill can also be loaded by Claude automatically when relevant
Explanation:
Custom commands have been merged into skills: a file at claude/commands/deploy-check.md and a skill at .claude/skills/deploy-check/SKILL.md both create /deploy-check and work the same way. Skills add a folder for supporting files, frontmatter that controls whether you or Claude invokes them, and automatic loading when the task matches the description. Existing claude/commands/ files keep working.

31. An architect is told 'multi-agent systems are always better than single-agent.'

Which response best reflects Anthropic's actual guidance?

A. Multi-agent is forbidden in production
B. Multi-agent adds coordination cost and complexity; use it only when a single agent or workflow cannot handle the task, and prefer the simplest design that works
C. Single LLM calls can never solve real tasks
D. Agree; always split into many agents for any task

Correct:
Multi-agent adds coordination cost and complexity; use it only when a single agent or workflow cannot handle the task, and prefer the simplest design that works
Explanation:
Anthropic's consistent theme is to use the simplest architecture that meets requirements and add complexity only when justified by evals. Multi-agent systems introduce coordination overhead, more tokens, and harder debugging, so they should be reserved for tasks that genuinely need them. The blanket claim that more agents are always better is incorrect.

32. Where should an architect place stable, authoritative instructions that define the assistant's role, rules, and persona for a production application?

A. In the last user message, every time
B. In a stop sequence
C. In the system prompt
D. In the tool input_schema description only

Correct:
In the system prompt
Explanation:
The system prompt is the right place for stable, authoritative instructions: role, rules, constraints, and persona. It carries higher standing than user-turn content and is reused across turns. Keeping authoritative directives in the system prompt also strengthens defences against injected user content.

33. An architect is asked to summarize the overarching reliability philosophy for building Claude-powered systems. Which statement best captures Anthropic's guidance?

A. Avoid evals because they slow shipping
B. Start simple, measure with evals, add complexity (tools, workflows, agents) only when justified, and bound autonomous behaviour with guardrails and stopping conditions
C. Always let agents act without any human oversight
D. Maximize architectural complexity up front to be future-proof

Correct:
Start simple, measure with evals, add complexity (tools, workflows, agents) only when justified, and bound autonomous behaviour with guardrails and stopping conditions
Explanation:
Anthropic's reliability philosophy is to start with the simplest effective solution, measure with comprehensive evals, increase complexity only when it demonstrably helps, and bound autonomous agents with guardrails, permissions, and clear stopping conditions. This keeps systems transparent, cost-effective, and safe. Complexity for its own sake and unbounded autonomy are anti-patterns.

34. A team building an agent on a model with adaptive thinking (e.g., Claude Opus 5.5 or Sonnet 4.6) wants Claude to reason between tool calls across a multi-step turn. What is true about enabling this interleaved thinking?

A. Interleaved thinking is impossible; Claude can only think once before the first tool call
B. Interleaved thinking requires manually setting a fixed budget_tokens value
C. Adaptive thinking automatically enables interleaved thinking, so Claude can reason between tool calls within a single assistant turn with no separate beta header
D. You must pass a separate interleaved-thinking beta header on every request

Correct:
Adaptive thinking automatically enables interleaved thinking, so Claude can reason between tool calls within a single assistant turn with no separate beta header
Explanation:
On models with adaptive thinking, interleaved thinking is automatic, letting Claude reason between tool calls within one assistant turn. No beta header is required and there is no fixed budget_tokens to set; depth is steered with the effort parameter. Thinking is already on for models such as Claude Opus 5.5, while Claude Opus 4.8 and Sonnet 4.6 turn it on with thinking type 'adaptive'. This suits agentic loops where each step depends on the previous tool result.

35. An architect is deciding how granular to make tools for an agent. Which design guidance leads to more reliable tool use?

A. Give every tool the same generic description
B. Create one giant do-everything tool with dozens of optional parameters
C. Design focused, well-named tools with clear descriptions and minimal, well-typed parameters, so the model can choose correctly
D. Hide the parameters from the schema to keep it short

Correct:
Design focused, well-named tools with clear descriptions and minimal, well-typed parameters, so the model can choose correctly
Explanation:
Focused tools with clear names, descriptions, and minimal well-typed parameters help the model select and call the right tool reliably. A single overloaded tool with many optional parameters increases ambiguity and error rates. Clear, distinct tool interfaces are effectively prompt engineering for the action surface.

36. A team mixes up two cache durations. Which statement about Anthropic prompt caching durations is correct?

A. The only available duration is 1 hour
B. The default cache lifetime is 24 hours
C. The default ephemeral cache lifetime is 5 minutes, with an optional 1-hour duration available at additional cost
D. Caches never expire

Correct:
The default ephemeral cache lifetime is 5 minutes, with an optional 1-hour duration available at additional cost
Explanation:
The supported cache type is ephemeral, with a default 5-minute lifetime, and an optional 1-hour duration is available at additional cost. The 24-hour figure refers to the structured-outputs schema compilation cache, not prompt caching. Keeping these straight avoids wrong cost and freshness assumptions.

37. An architect wants to guarantee that on a given turn Claude calls a specific tool named lookup_order, not just any tool. Which tool_choice configuration achieves this?

A. tool_choice set to type tool with name lookup_order
B. Btool_choice any
C. Ctool_choice auto
D. tool_choice none

Correct:
tool_choice set to type tool with name lookup_order
Explanation:
Setting tool_choice to type 'tool' with the specific name forces Claude to call exactly that tool (lookup_order) on the turn. 'auto' leaves it optional, 'any' forces some tool but not a specific one, and 'none' forbids tools. Forced tool use is not available on every model or setting: with manual extended thinking, and on Claude Opus 5.5, Sonnet 5.5, Fable 5.1 and Mythos 5.1, 'any' and 'tool' return an error, so there you use auto with strict tool use and prompt for the tool.

38. A nightly job must process 200,000 classification requests where results are not needed for hours. Which approach minimizes cost while meeting the latency requirement?

A. Use the Message Batches API, which processes asynchronously within 24 hours at a 50% discount on input and output tokens
B. Increase temperature to speed processing
C. Use extended thinking on every request
D. Send all requests synchronously at full price for instant results

Correct:
Use the Message Batches API, which processes asynchronously within 24 hours at a 50% discount on input and output tokens
Explanation:
The Message Batches API processes large volumes asynchronously (within 24 hours) at a 50% discount on both input and output tokens, ideal when results are not needed immediately. It also stacks with prompt caching for even greater savings. Synchronous calls would cost twice as much for no benefit here.

39. An architect wants to add an MCP server to Claude Code that loads only in the current project, stays private to one developer, and is not committed to the shared repository. Which scope should they use?

A. --scope project, stored in .mcp.json at the project root
B. It is impossible to limit an MCP server to a single developer
C. --scope user, stored in ~/.claude.json and loaded in all of that developer's projects
D. --scope local (the default), stored in ~/.claude.json under that project's path

Correct:
--scope local (the default), stored in ~/.claude.json under that project's path
Explanation:
Local scope is the default for claude mcp add. A local-scoped server loads only in the project where it was added and stays private to the developer; Claude Code stores it in ~/.claude.json under that project's path. It is not stored in .claude/settings.local.json, which holds general local settings. Project scope writes .mcp.json at the project root, which is meant to be committed and shared, and user scope also lives in ~/.claude.json but loads in every project on the machine.

40. In a workflow, the output of step 1 is transformed and fed as input to step 2, whose output feeds step 3. This decomposition trades latency for accuracy on each subtask. Which pattern is this and when is it ideal?

A. Orchestrator-worker, ideal for unpredictable subtasks
B. Voting, ideal for consensus
C. Prompt chaining, ideal when a task cleanly decomposes into fixed sequential subtasks
D. Routing, ideal for category dispatch

Correct:
Prompt chaining, ideal when a task cleanly decomposes into fixed sequential subtasks
Explanation:
Prompt chaining decomposes a task into a fixed sequence where each step's output feeds the next, trading added latency for higher accuracy on each focused subtask. It is ideal when the decomposition is known and stable. Programmatic gates can sit between steps to catch errors early.

41. A team enables prompt caching but their small system prompt never produces a cache hit on Sonnet. What is the most likely cause?

A. Caching is broken in the API
B. The cached prefix is below the model's minimum cacheable prompt length (e.g., 1,024 tokens on Claude Sonnet 4.6), so it is processed without caching
C. They must set temperature to O for caching to work
D. Caching only works with batch requests

Correct:
The cached prefix is below the model's minimum cacheable prompt length (e.g., 1,024 tokens on Claude Sonnet 4.6), so it is processed without caching
Explanation:
Prompt caching has a minimum cacheable prompt length that varies by model: for example 512 tokens on Claude Sonnet 5.5, Opus 5.5 and Haiku 5.5, 1,024 on Sonnet 4.6, and 4,096 on Haiku 4.5. A shorter prompt is processed without caching and no error is returned; cache_creation_input_tokens and cache_read_input_tokens both stay at 0. The fix is to make the cached prefix reach the model's minimum.

42. A team wants parallelization but for a different reason than latency: they want multiple independent perspectives on the same risky decision, then a vote. Which form of parallelization is this?

A. Routing, where each run handles a different category
B. Voting, where the same task is run multiple times to get diverse outputs and aggregate by consensus
C. Sectioning, where one task is split into independent subtasks
D. Chaining, where each run depends on the previous

Correct:
Voting, where the same task is run multiple times to get diverse outputs and aggregate by consensus
Explanation:
Anthropic describes two flavours of parallelization: sectioning (splitting a task into independent subtasks) and voting (running the same task multiple times to get diverse outputs and aggregating, often by majority or threshold). Voting improves confidence on high-stakes or error-prone judgments. The runs are independent and then combined.

43. A large repo causes Claude Code to waste turns rediscovering structure. Which CLAUDE.md content gives the highest leverage for efficiency and correctness?

A. A full copy of every source file pasted inline
B. Concise, durable rules: key directories, build/test commands, conventions, and architecture constraints that shape most decisions
C. Nothing; CLAUDE.md should be left empty so Claude explores freely
D. A list of unrelated personal preferences

Correct:
Concise, durable rules: key directories, build/test commands, conventions, and architecture constraints that shape most decisions
Explanation:
An effective CLAUDE.md is lean and high-signal: it captures the rules and context that shape most decisions, such as key directories, how to build and test, conventions, and hard architectural constraints. This anchors behaviour and avoids wasted rediscovery, while staying small enough to not bloat context. Dumping entire files or noise is counterproductive.

44. An architect needs to decide which model to use for a task requiring the full 1M-token context window. Which statement reflects current model context capabilities?

A. Context window size is configurable to any value by the caller
B. Many models (e.g., Claude Opus 5.5, Sonnet 5.5, Haiku 5.5 and Fable 5.1) have a 1M-token context window, while some others (e.g., Claude Haiku 4.5 and Sonnet 4.5) are limited to 200K tokens
C. No Claude model exceeds 200K tokens
D. All Claude models have a 1M-token context window

Correct:
Many models (e.g., Claude Opus 5.5, Sonnet 5.5, Haiku 5.5 and Fable 5.1) have a 1M-token context window, while some others (e.g., Claude Haiku 4.5 and Sonnet 4.5) are limited to 200K tokens
Explanation:
Context window size is a per-model property. Anthropic's context-window docs list a 1M-token window for models including Claude Fable 5.1, Opus 5.5, Sonnet 5.5, Haiku 5.5, Opus 4.8 and Sonnet 4.6, while other models, such as Claude Sonnet 4.5 (deprecated) and Haiku 4.5, have 200K. An architect who needs 1M tokens must pick a model that supports it, and can check each model's limit (max_input_tokens) with the Models API.

45. A tool that fetches data from a flaky upstream sometimes times out. From a tool-design perspective, how should the tool report this so the agent can recover well?

A. Return a clear error result (e.g., tool_result with is_error true and a concise message) so the model can decide to retry or take another path
B. Throw away the request silently
C. Return a 200 with empty data as if it succeeded
D. Return a 5,000-line stack trace as the result

Correct:
Return a clear error result (e.g., tool_result with is_error true and a concise message) so the model can decide to retry or take another path
Explanation:
Tools should surface failures honestly with a concise, actionable error (for client tools, a tool_result with is_error true), so the agent can reason about retrying or routing around the failure. Faking success or hiding the error corrupts the agent's reasoning. A giant stack trace wastes context; a concise message is better.

46. A team's Claude Code sessions keep losing earlier context on long tasks. They want a session to resume exactly where it left off in a later run. In the Agent SDK /Claude Code session model, how is this achieved?

A. Every run is stateless and resuming is impossible
B. Capture the session_id from the init message and pass it back (resume) on the next call to continue the same session
C. Increase max_tokens to retain memory
D. Paste the entire prior transcript into CLAUDE.md

Correct:
Capture the session_id from the init message and pass it back (resume) on the next call to continue the same session
Explanation:
By default each query starts a fresh session, but capturing the session_id from the init message and passing it back as the resume parameter continues exactly where you left off. This is the supported mechanism for persistent, resumable sessions. It avoids manual transcript copying.

47. A coding task requires generating a draft, then critiquing it against quality criteria, then refining it, repeating until the critique passes. Which pattern describes this, and what is the key design principle?

A. Orchestrator-worker, with one orchestrator spawning all critics at once
B. Parallelization, generating many drafts and averaging them
C. Routing, where a classifier picks the best draft from several
D. Evaluator-optimizer, separating generation and evaluation into distinct LLM calls with an adversarial evaluator stance

Correct:
Evaluator-optimizer, separating generation and evaluation into distinct LLM calls with an adversarial evaluator stance
Explanation:
The evaluator-optimizer pattern uses one call to generate and another to evaluate and give feedback, looping until criteria are met. Anthropic recommends separating generation from evaluation into distinct calls and giving the evaluator an adversarial stance so it finds real problems. This is one of the highest-leverage improvements for quality-sensitive tasks.

48. An architect wants the model to track how much context it has left during a long task so it can manage its work. Which model capability supports this?

A. Prompt caching
B. Stop sequences
C. Context awareness, which some models (such as Claude Sonnet 5, Sonnet 4.6 and Haiku 4.5) use to track their remaining context window
D. Strict tool use

Correct:
Context awareness, which some models (such as Claude Sonnet 5, Sonnet 4.6 and Haiku 4.5) use to track their remaining context window
Explanation:
Context awareness lets the models that have it (Claude Sonnet 5, Sonnet 4.6, Sonnet 4.5 and Haiku 4.5) track their remaining context window, their token budget, throughout a conversation, so they can manage a long task against the space left. It is automatic on those models. Newer models such as Claude Opus 5.5, Sonnet 5.5 and Haiku 5.5 don't receive these budget tags; there you can set an explicit budget with task budgets (beta). It is distinct from caching or output constraints.

49. An architect wants to distinguish a 'workflow' from an 'agent' in Anthropic's terminology. Which statement is correct?

A. Workflows orchestrate LLMs and tools through predefined code paths; agents dynamically direct their own processes and tool usage
B. Agents never use tools, while workflows always do
C. Workflows let the LLM dynamically direct its own process and tool usage; agents follow predefined code paths
D. Workflows and agents are identical terms with no practical difference

Correct:
Workflows orchestrate LLMs and tools through predefined code paths; agents dynamically direct their own processes and tool usage
Explanation:
Anthropic defines workflows as systems where LLMs and tools are orchestrated through predefined code paths, while agents are systems where the LLM dynamically directs its own processes and tool usage, controlling how it accomplishes the task. The distinction guides architecture choices: workflows offer predictability, agents offer flexibility for open-ended problems.

50. A team builds an agent whose outputs feed a downstream automated system requiring strict JSON. They want the agent to act autonomously but still produce machine-parseable final output. What is the cleanest architectural approach?

A. Let the agent return free-form prose and parse it with brittle regex
B. Disable tools so the agent cannot deviate
C. Ask the agent politely in the prompt and hope it complies every time
D. Constrain the final response with structured outputs (output_config.format with a JSON schema) or strict tool use, so the machine-readable contract is guaranteed

Correct:
Constrain the final response with structured outputs (output_config.format with a JSON schema) or strict tool use, so the machine-readable contract is guaranteed
Explanation:
When a downstream system needs guaranteed-valid JSON, you enforce it with structured outputs (the output_config.format parameter compiling a JSON schema into a grammar) or strict tool use (strict: true on the tool). This makes conformance a hard contract rather than a hope. The agent can still reason and use tools while its final emission is constrained.

51. A team's CLAUDE.md contains hard rules like 'do not push to GitHub unless explicitly asked.' Why is encoding such rules in CLAUDE.md valuable architecturally?

A. It encrypts the repository
B. It persistently steers the agent's default behaviour every session, reducing reliance on the user repeating constraints
C. It speeds up the model's inference
D. It guarantees the underlying Git server rejects pushes

Correct:
It persistently steers the agent's default behaviour every session, reducing reliance on the user repeating constraints
Explanation:
Durable behavioural constraints in CLAUDE.md are reloaded each session, so they steer the agent's defaults consistently without the user repeating them. This improves predictability and reduces the chance of unwanted actions. It is guidance enforced by being in the agent's standing context, not a server-side control.

52. When integrating a third-party MCP server an architect did not write, which security consideration is most important before granting it tool access in production?

A. Whether it uses stdio or HTTP, since both are equally trustworthy regardless of source
B. Whether the server has many tools, since more tools is always safer
C. Trust and scope: vet the server, apply least-privilege permissions/allowlists, and consider that a malicious or compromised server's tools/resources could attempt prompt injection or data exfiltration
D. Whether the server's logo matches the brand

Correct:
Trust and scope: vet the server, apply least-privilege permissions/allowlists, and consider that a malicious or compromised server's tools/resources could attempt prompt injection or data exfiltration
Explanation:
A third-party MCP server runs code and can return content the model treats as context, so it must be vetted and scoped with least-privilege permissions; a malicious server could attempt injection or exfiltration through its tools or resources. Allow/deny rules and limited tool grants reduce blast radius. Trust and scope, not cosmetics, drive the decision.

53. To improve a classifier's accuracy, an architect adds three carefully chosen labelled examples directly in the prompt. What is this technique called and why does it help?

A. Few-shot prompting, which demonstrates the desired input-output mapping in context to guide the model
B. Fine-tuning, because it updates model weights
C. Prompt caching, because examples are cached
D. Routing, because examples route the request

Correct:
Few-shot prompting, which demonstrates the desired input-output mapping in context to guide the model
Explanation:
Providing example input-output pairs in the prompt is few-shot prompting; it demonstrates the desired mapping and format so the model generalizes the pattern. It is a fast, weight-free way to improve accuracy and consistency. Anthropic's best practices recommend well-chosen examples for complex tasks.

54. An architect wants Claude's tool calls to always have parameters matching the tool's input_schema exactly. Which setting enforces this?

A. Removing the input_schema
B. Setting tool_choice to none
C. Adding strict: true to the tool definition (strict tool use)
D. Increasing max_tokens

Correct:
Adding strict: true to the tool definition (strict tool use)
Explanation:
Strict tool use adds strict: true to a tool definition so Claude's tool-call parameters are guaranteed to match the tool's input_schema exactly. This prevents malformed arguments reaching your function. It complements JSON output mode, which constrains the assistant's text response.

55. An architect must keep latency low and cost predictable for an interactive assistant while preserving a long, stable instruction set. Which design balances these best?

A. Send the long instructions fresh, uncached, on every turn
B. Cache the stable instruction prefix with prompt caching and choose a fast model tier (e.g., Sonnet/Haiku) appropriate to the task difficulty
C. Remove the instructions to save tokens
D. Use the largest Opus model with extended thinking on every trivial turn

Correct:
Cache the stable instruction prefix with prompt caching and choose a fast model tier (e.g., Sonnet/Haiku) appropriate to the task difficulty
Explanation:
Caching the stable instruction prefix makes repeated turns cheap and fast, and matching the model tier to task difficulty (Haiku/Sonnet for routine interactive turns) keeps latency and cost predictable. Sending uncached instructions every turn or using Opus-plus-thinking for trivial turns wastes both. This pairs caching with right-sized model selection.

56. A team wants Claude Code to operate in a mode where it can read and plan but cannot make edits or run commands until they review the plan. Which configuration concept supports this?

A. Increasing the thinking budget
B. Setting difficulty to hard
C. Disabling the model entirely
D. Permission modes (e.g., a plan/read-only mode) that restrict which actions the agent may take

Correct:
Permission modes (e.g., a plan/read-only mode) that restrict which actions the agent may take
Explanation:
Claude Code and the Agent SDK support permission modes that constrain what actions are allowed, such as a planning or read-only mode where the agent can analyse but not edit or execute until approved. This lets a human review the plan before any changes occur. It is a core safety/workflow control.

57. MCP is often said to solve the 'N times M' integration problem. What does this mean architecturally?

A. It requires N servers for M models with no reuse
B. It forces every model to be retrained for each tool
C. It multiplies token costs by N times M
D. It standardizes how Al applications connect to external systems, so you avoid building custom point-to-point integrations between every model and every tool

Correct:
It standardizes how Al applications connect to external systems, so you avoid building custom point-to-point integrations between every model and every tool
Explanation:
MCP provides a standardized protocol so any compliant client can talk to any compliant server, eliminating the need to build a custom integration for every model-tool pair (the N times M problem). One server can serve many clients and one client can use many servers. This reuse is the core architectural value of MCP.

58. A production agent must recover gracefully when a tool call fails (timeout or 5xx). What is the most robust architectural approach to surface the failure to the model?

A. Silently drop the failed call and continue as if it succeeded
B. Return a tool_result block with is_error set to true so Claude can see the failure and decide how to retry or adapt
C. Convert the error into a fake successful result
D. Crash the whole agent process immediately

Correct:
Return a tool_result block with is_error set to true so Claude can see the failure and decide how to retry or adapt
Explanation:
When a client tool errors, you return a tool_result content block with is_error: true containing the error detail, so Claude can observe what failed and adapt, such as retrying or choosing a different approach. Hiding the error or faking success leads the model to reason on false premises. This pattern keeps the agent loop honest and recoverable.

59. An architect wants to reduce the chance Claude calls a tool with malformed arguments that crash the downstream function. Combining which two measures is most effective?

A. A precise input_schema plus strict tool use (strict: true) so parameters are validated to match exactly
B. Disabling tools and answering from memory
C. Removing the schema and trusting the model
D. A vague description plus high temperature

Correct:
A precise input_schema plus strict tool use (strict: true) so parameters are validated to match exactly
Explanation:
A precise input_schema defines exactly what arguments are valid, and strict tool use (strict: true) guarantees Claude's calls conform to that schema. Together they prevent malformed arguments reaching your function. This pairs interface clarity with hard enforcement.

60. A team needs the model to choose freely whether to use a tool or just answer in plain text. Which tool_choice setting expresses this?

A. any
B. none
C. tool
D. auto

Correct:
auto
Explanation:
tool_choice: auto lets Claude decide whether to call a tool or respond directly with text, which is the default flexible behaviour. 'any' forces some tool call, 'tool' forces a specific named tool, and 'none' forbids tools. Selecting the right tool_choice is core to controlling agent behaviour.

61. A CI pipeline needs Claude Code to run non-interactively to triage failing tests and post a summary, with no human at a terminal. Which usage mode fits?

A. Headless (non-interactive) mode, suitable for automation and CI
B. Only the web chat interface
C. Interactive REPL mode only
D. It cannot run without a human present

Correct:
Headless (non-interactive) mode, suitable for automation and CI
Explanation:
Claude Code supports headless, non-interactive usage designed for automation and CI, where it runs a task and returns output without an interactive session. This is how teams integrate it into pipelines and scripts. Pairing it with permission rules and hooks keeps automated runs bounded.

62. A team measures whether their agent actually improved after adding an evaluator-optimizer loop. According to Anthropic, what should anchor this decision?

A. Subjective impressions from a single demo run
B. The number of tools calls the agent makes
C. Whether the agent uses the newest model
D. A systematic evaluation (evals) measuring task success on representative cases before and after the change

Correct:
A systematic evaluation (evals) measuring task success on representative cases before and after the change
Explanation:
Anthropic stresses building agents on a foundation of measurement: optimize prompts with comprehensive evaluation and add complexity only when evals show it helps. To justify an evaluator-optimizer loop you compare task-success metrics on representative cases before and after. Without evals you cannot tell whether added complexity actually improved outcomes.

63. A pipeline uses structured outputs and notices the schema is compiled and cached. According to the docs, for how long is a compiled schema cached?

A. 1 hour
B. 5 minutes
C. 24 hours
D. It is never cached and recompiled every token

Correct:
24 hours
Explanation:
When you send a request with structured outputs (output_config.format), Claude compiles your JSON schema into a grammar and caches that compiled schema for 24 hours, then applies the constraint while generating each token. The 24-hour cache avoids recompiling the same schema on every request. This is distinct from prompt caching's default 5-minute lifetime.

64. A production agent occasionally hits transient API errors (rate limits, 529 overloaded). Which reliability pattern is most appropriate at the client level?

A. Immediately give up on the first error
B. Resend the request thousands of times instantly in a tight loop
C. Retry with exponential backoff and jitter, respecting rate-limit signals, before failing
D. Switch to a smaller context window

Correct:
Retry with exponential backoff and jitter, respecting rate-limit signals, before failing
Explanation:
Transient errors like rate limits or overload are best handled with retries using exponential backoff and jitter, honoring any retry guidance, before surfacing a failure. This smooths over temporary conditions without hammering the API. Immediate give-up or tight-loop retries are both poor reliability choices.

65. An architect repeatedly sends the same large system prompt and tool definitions on every request. Which Claude API feature reduces cost and latency for the unchanging prefix?

A. Prompt caching with cache_control, which caches the prefix (tools, system, messages in that order) so subsequent reads are far cheaper
B. Raising max_tokens
C. Switching to stop sequences
D. Disabling the system prompt

Correct:
Prompt caching with cache_control, which caches the prefix (tools, system, messages in that order) so subsequent reads are far cheaper
Explanation:
Prompt caching marks a prefix with cache_control so repeated requests reuse it: caching references the entire prompt up to the marked block in the order tools, system, then messages. Cache reads cost about 0.1x base input price, dramatically cutting cost and latency for stable prefixes. This is ideal for large, unchanging system prompts and tool definitions.

66. An architect wants a Claude Code hook to make a yes/no decision by asking a model (not a shell command) whether a change is safe before allowing it. Which hook handler type fits?

A. A command hook running a shell script
B. A Stop hook that ends the session
C. A prompt hook (type: prompt) that sends a single-turn prompt to a model returning a yes/no decision
D. An mcp_tool hook that uploads a file

Correct:
A prompt hook (type: prompt) that sends a single-turn prompt to a model returning a yes/no decision
Explanation:
A prompt hook sends a single-turn prompt to a Claude model for evaluation and returns a yes/no decision as JSON, which is ideal for lightweight model-judged gating. Command hooks run shell commands, and mcp_tool hooks call a connected MCP server tool. Choosing the handler type to match the decision logic is part of designing reliable hooks.

67. A team wants a shell command to run automatically every time Claude Code is about to use the Edit tool, to enforce a check. Which Claude Code feature provides this lifecycle interception?

A. Renaming the Edit tool
B. CLAUDE.md prose instructions
C. Hooks, configured in settings with an event and a matcher
D. Slash commands

Correct:
Hooks, configured in settings with an event and a matcher
Explanation:
Hooks are user-defined handlers (shell commands, HTTP endpoints, or prompts) that fire at specific lifecycle events such as PreToolUse, with a matcher to filter which tools trigger them. A PreToolUse hook matching Edit runs deterministically before each edit, which prose in CLAUDE.md cannot guarantee. Hooks give programmatic, enforced control rather than best-effort instructions.

68. A tool returns a very large payload that would consume most of the context window. What is the best tool-design practice to keep the agent reliable?

A. Return concise, relevant results (with pagination, filtering, or summaries) so the tool output does not overwhelm the context window
B. Return nothing and let the model guess
C. Always return the entire raw payload regardless of size
D. Encode the payload as a single unbroken string with no structure

Correct:
Return concise, relevant results (with pagination, filtering, or summaries) so the tool output does not overwhelm the context window
Explanation:
Tools should return concise, relevant output, using pagination, filtering, or summarization so a single result does not flood the context window and degrade performance. Dumping huge raw payloads wastes context and can crowd out the task. Designing tool outputs for the model's context budget is a core reliability practice.

69. Which scenario is the clearest signal that a task genuinely needs an autonomous agent rather than a fixed workflow?

A. The task is a single deterministic database lookup
B. The task is a one-shot translation with no tools
C. The task always follows the exact same three steps every time
D. The number and order of steps cannot be predicted in advance and depend on what the model discovers as it works

Correct:
The number and order of steps cannot be predicted in advance and depend on what the model discovers as it works
Explanation:
Autonomous agents are warranted when the path to a solution is open-ended: the steps, their order, and the tools needed cannot be predetermined and must be decided dynamically as the model gathers information. Fixed, predictable tasks are better served by workflows or even a single call. Matching architecture to predictability is a core CCA-F skill.

70. A prompt mixes a long, untrusted document with the task instructions. To reduce the risk that text inside the document hijacks the instructions, what prompt-engineering practice helps most?

A. Put the instructions inside the document so they travel together
B. Remove all delimiters so the model treats everything uniformly
C. Paste the document into the system prompt as instructions
D. Clearly separate and label the untrusted document (e.g., within tags) and keep the actual instructions outside it, ideally in the system prompt

Correct:
Clearly separate and label the untrusted document (e.g., within tags) and keep the actual instructions outside it, ideally in the system prompt
Explanation:
Clearly delimiting and labelling untrusted content (for example inside tags) while keeping authoritative instructions separate, ideally in the system prompt, helps the model treat the document as data rather than commands. This is a key defence against prompt injection. Blurring the boundary or elevating the document to instruction status increases risk.

71. An agent autonomously calls tools in a loop. Anthropic recommends giving the agent guardrails and a way to stop. Which design element most directly bounds runaway loops in a production agent?

A. Removing the system prompt to reduce token cost
B. Disabling all tool use after the first call
C. Increasing max_tokens to the model's maximum on every call
D. Setting a maximum number of turns/iterations and clear stopping conditions

Correct:
Setting a maximum number of turns/iterations and clear stopping conditions
Explanation:
Autonomous agents run in a loop, so you must bound them with stopping conditions and a maximum number of turns to prevent runaway cost or infinite loops. The Agent SDK exposes options like maxTurns for exactly this. Clear stop criteria are a core reliability guardrail for agentic systems.

72. MCP defines three core server primitives. Which set correctly names them and their roles?

A. Tools (executable functions), Resources (contextual data at a URI), Prompts (reusable templates)
B. Models, Tokens, Embeddings
C. Routes, Controllers, Views
D. Hooks, Sessions, Permissions

Correct:
Tools (executable functions), Resources (contextual data at a URI), Prompts (reusable templates)
Explanation:
MCP servers expose three core primitives: tools (executable functions the client can invoke to take actions), resources (data sources exposed at a URI that provide context), and prompts (reusable templates that structure interactions). Knowing these distinctions is fundamental to designing MCP integrations. Tools compute and act, resources supply data, and prompts package ready-to-use context.

73. An architect designs an MCP tool that deletes records. Following MCP security best practices, what is the most important safeguard?

A. Treat tool invocation as trusted and skip any authorization
B. Apply least privilege and require authorization/confirmation for destructive operations, validating inputs server-side
C. Expose the database admin credentials in the tool description
D. Allow any client to call it without authentication

Correct:
Apply least privilege and require authorization/confirmation for destructive operations, validating inputs server-side
Explanation:
MCP servers should enforce least privilege, validate inputs, and require authorization or confirmation for destructive actions, since tools execute real operations. Destructive tools especially need guarding because a model can be wrong or manipulated. Server-side validation and authz are the server's responsibility, not the model's.

74. A developer adds an MCP server to Claude Code and wants its tools, resources, and prompts available. After configuration, how does the client typically discover what the server offers?

A. It calls the server's list methods (e.g., tools/list, resources/list, prompts/list) to discover available primitives
B. It guesses based on the server name
C. The developer must hard-code every tool name in CLAUDE.md
D. Discovery is impossible; you must restart for each tool

Correct:
It calls the server's list methods (e.g., tools/list, resources/list, prompts/list) to discover available primitives
Explanation:
MCP clients discover a server's capabilities using its list methods, such as tools/list, resources/list, and prompts/list, then invoke the chosen primitive. This dynamic discovery is core to MCP's design and avoids hard-coding. It lets one client work with many servers without bespoke integration.

75. An architect must extract structured data AND have the model call a tool with guaranteed-valid parameters in the same workflow. Which combination is supported?

A. Only message prefilling can do both
B. Neither can be used together under any circumstances
C. Structured outputs disable tool use entirely
D. JSON outputs mode combined with strict tool use, so the response is valid JSON and tool calls match their schemas

Correct:
JSON outputs mode combined with strict tool use, so the response is valid JSON and tool calls match their schemas
Explanation:
JSON outputs mode (controlling what Claude says) can be combined with strict tool use (validating how Claude calls your functions), so you get guaranteed-valid JSON responses and schema-conformant tool calls together. Structured outputs also work with batch, token counting, and streaming. They do not, however, work with prefilling or citations.

76. A team uses the Claude Agent SDK and spawns several subagents in parallel for a large refactor. Each subagent does deep analysis. What is the main architectural advantage of using separate subagents here?

A. Subagents cannot use tools, which makes them faster
B. Each subagent gets its own context window, so one subagent's exploration does not pollute the others' or the main context
C. Subagents automatically inherit all parent permissions, removing any approval prompts
D. Subagents share one context window, so all analysis is visible to every agent

Correct:
Each subagent gets its own context window, so one subagent's exploration does not pollute the others' or the main context
Explanation:
In the Claude Agent SDK, each subagent runs with its own isolated context window, so verbose exploration in one subagent does not consume or pollute the main conversation or sibling subagents. This context preservation is a primary reason to use subagents for parallelizable, context-heavy subtasks. The main agent can then synthesize concise results.

77. A data-extraction service must return JSON that always conforms to a fixed schema, with no extra prose, parseable directly from the response. Which Claude API feature provides a hard guarantee of schema conformance?

A. Politely asking for JSON in the prompt
B. Lowering temperature to O
C. Structured outputs via the output_config.format parameter, which compiles your JSON schema into a grammar that constrains generation
D. Setting a stop sequence of '}'

Correct:
Structured outputs via the output_config.format parameter, which compiles your JSON schema into a grammar that constrains generation
Explanation:
Structured outputs use the output_config.format parameter (the canonical replacement for the older output_format parameter) to compile your JSON schema into a grammar that constrains every generated token, yielding guaranteed-valid JSON in the response. Prompting, stop sequences, or temperature only nudge behaviour and can still produce invalid output. The grammar constraint is what makes conformance a hard guarantee.

78. An architect needs to classify incoming requests and send each to a specialized prompt: billing questions to one prompt, technical issues to another, and general questions to a third. Which workflow pattern best fits this requirement?

A. Parallelization
B. Prompt chaining
C. Evaluator-optimizer
D. Routing

Correct:
Routing
Explanation:
Routing classifies an input and directs it to a specialized downstream prompt or model. It works well when there are distinct categories better handled separately and when classification can be done accurately. Separating concerns this way lets you optimize each path independently without one prompt trying to handle everything.

79. When designing a multi-step agent, Anthropic emphasizes 'showing the agent's planning steps' and keeping the design simple. What is the primary architectural benefit of this transparency?

A. It makes the agent's behaviour easier to debug, audit, and trust, and easier to course-correct
B. It eliminates the need for any evaluation or testing
C. It allows the model to skip tool calls entirely
D. It guarantees the agent never makes a mistake

Correct:
It makes the agent's behaviour easier to debug, audit, and trust, and easier to course-correct
Explanation:
Anthropic recommends prioritizing transparency by explicitly showing the agent's planning steps. Exposing the plan makes behaviour observable, so engineers can debug, audit, and intervene when the agent goes off track. Transparency is a core operability principle for production agents, not a correctness guarantee.

80. An architect must connect Claude to a local MCP server running as a child process on the same machine for lowest overhead. Which transport is most appropriate?

A. Raw UDP multicast
B. Stdio transport, using standard input/output streams between local processes
C. Streamable HTTP transport over the public internet
D. Email

Correct:
Stdio transport, using standard input/output streams between local processes
Explanation:
Stdio transport uses standard input/output streams for direct communication between local processes on the same machine, with no network overhead, making it ideal for local servers. Streamable HTTP is for remote servers and supports HTTP auth. Choosing the transport to match local vs remote deployment is a key MCP design decision.

81. A latency-sensitive content pipeline must run a translation, a sentiment analysis, and a toxicity check on the same input, all independent of each other, then combine the results. Which pattern minimizes end-to-end latency?

A. Prompt chaining the three checks sequentially
B. Evaluator-optimizer looping over each check
C. Parallelization (sectioning), running the three independent calls concurrently and aggregating
D. A single giant prompt asked to do all three plus self-grade

Correct:
Parallelization (sectioning), running the three independent calls concurrently and aggregating
Explanation:
Parallelization runs independent subtasks simultaneously and aggregates their outputs. When subtasks do not depend on one another, sectioning them into concurrent calls reduces wall-clock latency compared to sequential chaining. It can also improve focus, since each call handles one concern.

82. A document-processing pipeline must (1) extract structured fields, (2) validate them against business rules, and (3) only then summarize. Step 2 acts as a programmatic gate. What is this gate called in prompt chaining, and why use it?

A. A 'router' that randomly selects which step runs next
B. An 'orchestrator' that spawns workers for each field
C. A 'parallel branch' that runs all three steps at once
D. A 'guardrail' or programmatic check between steps that catches errors early before they propagate downstream

Correct:
A 'guardrail' or programmatic check between steps that catches errors early before they propagate downstream
Explanation:
In prompt chaining you can insert programmatic checks (gates or guardrails) between steps to verify intermediate output before continuing. Catching a malformed extraction at step 2 prevents wasted work and bad summaries downstream. This trades a small amount of latency for substantially higher reliability.

83. A PreToolUse hook should automatically approve a specific safe tool so subagents do not repeatedly prompt for permission. Why are hooks well suited to this versus relying on the model?

A. Hooks are suggestions the model may ignore
B. Hooks only run after a tool already executed
C. Hooks require the user to click approve every time anyway
D. Hooks execute deterministically at the lifecycle event and can return an approve/deny decision, applying even to subagent sessions

Correct:
Hooks execute deterministically at the lifecycle event and can return an approve/deny decision, applying even to subagent sessions
Explanation:
Hooks run as deterministic code at defined events and can emit decisions, so a PreToolUse hook can auto-approve a specific tool, including for subagent sessions that would otherwise each prompt. This is more reliable than model-dependent behaviour because it is enforced by the harness. Anthropic specifically recommends PreToolUse hooks or permission rules to avoid repeated subagent prompts.

84. Following Anthropic's prompting best practices, which technique most reliably improves accuracy on a complex extraction task?

A. Avoid examples because they bias the model
B. Always set temperature to its maximum
C. Use the shortest possible vague prompt and hope the model infers intent
D. Be explicit and specific about the desired output, and provide examples (few-shot) and clear structure

Correct:
Be explicit and specific about the desired output, and provide examples (few-shot) and clear structure
Explanation:
Anthropic's best practices emphasize being explicit and specific about the desired behaviour and output format, and providing well-chosen examples and structure. Clear instructions plus representative few-shot examples reduce ambiguity and improve reliability on complex tasks. Vague prompts and maxed-out randomness do the opposite.

85. A workflow chains: classify intent (Haiku) then, only for complex cases, draft a detailed answer (Opus). This combines two patterns. Which two?

A. Evaluator-optimizer plus compaction
B. Routing plus model selection (escalating hard cases to a stronger model)
C. Voting plus sectioning
D. Pure parallelization with no routing

Correct:
Routing plus model selection (escalating hard cases to a stronger model)
Explanation:
Classifying intent and sending only complex cases to a stronger model combines routing (branching by classification) with model selection (escalating difficulty to Opus while keeping cheap Haiku for the common path). Anthropic notes that production agents usually combine multiple patterns. This keeps average cost low while preserving quality where it matters.

86. A team wants the same architecture rules applied across multiple repositories for one developer, regardless of which project they open. Where should these personal cross-project rules live?

A. Nowhere; cross-project config is unsupported
B. In a random text file on the desktop
C. In each repo's committed.claude/settings.json
D. In the user-level ~/.claude/CLAUDE.md (or ~/.claude/rules/), which applies to all of that user's projects

Correct:
In the user-level ~/.claude/CLAUDE.md (or ~/.claude/rules/), which applies to all of that user's projects
Explanation:
User-level memory in ~/.claude/CLAUDE.md holds personal preferences for all projects and is not shared with teammates through version control, so it is the right place for one developer's cross-project rules. A repository's CLAUDE.md or committed .claude/settings.json applies only to that repository and to the whole team. Choosing scope by intended audience and breadth is a key configuration skill.

87. A team is building a customer-support agent. They want the simplest reliable architecture and only plan to add complexity if needed. According to Anthropic's guidance on building effective agents, what should they start with?

A. An orchestrator-worker system with at least five specialized workers from day one
B. The simplest solution possible, such as a single LLM call with retrieval and in-context examples, adding agentic steps only when simpler approaches fall short
C. A multi-agent swarm where dozens of agents negotiate the answer
D. A fine-tuned model dedicated to support before writing any prompts

Correct:
The simplest solution possible, such as a single LLM call with retrieval and in-context examples, adding agentic steps only when simpler approaches fall short
Explanation:
Anthropic's 'Building Effective Agents' guidance is explicit: find the simplest solution possible and only increase complexity when needed. Many use cases are solved by a single optimized LLM call with retrieval and examples, and full agentic systems should be reserved for when simpler patterns demonstrably fall short. This keeps systems transparent, cheaper, and easier to evaluate.

88. MCP's protocol is described as JSON-RPC based at the data layer. Why does this standardization matter for an architect choosing MCP?

A. It means each integration must be custom-coded per client
B. It removes the need for any authentication ever
C. A common JSON-RPC protocol for lifecycle and primitives (tools/resources/prompts/notifications) lets any compliant client and server interoperate without bespoke glue
D. It ties servers to a single proprietary vendor

Correct:
A common JSON-RPC protocol for lifecycle and primitives (tools/resources/prompts/notifications) lets any compliant client and server interoperate without bespoke glue
Explanation:
MCP's data layer defines a JSON-RPC protocol covering lifecycle management and core primitives (tools, resources, prompts, notifications), so any compliant client and server can interoperate. This standardization is what enables reuse across the ecosystem without writing custom glue per pairing. It is the foundation of MCP's portability.

89. An architect distinguishes client tools from server tools in the Claude API. Which statement is accurate?

A. Client tools run in your application and you return tool_result; server tools (like web_search) run on Anthropic's infrastructure and you receive results directly
B. Server tools require you to implement their execution yourself
C. Client tools always run on Anthropic's servers
D. There is no difference between client and server tools

Correct:
Client tools run in your application and you return tool_result; server tools (like web_search) run on Anthropic's infrastructure and you receive results directly
Explanation:
Client tools execute in your application: Claude emits tool_use, you run it, then return a tool_result. Server tools such as web_search, code_execution, and web_fetch run on Anthropic's infrastructure, so you see results without handling execution. The distinction is where the code runs and who executes it.

90. An agentic workflow accumulates many stale tool results in context, degrading performance. Which context-engineering strategy directly addresses this?

A. Increasing max_tokens
B. Forcing tool_choice to any
C. Tool result clearing (context editing) to remove old tool outputs no longer needed
D. Switching transports from stdio to HTTP

Correct:
Tool result clearing (context editing) to remove old tool outputs no longer needed
Explanation:
Context editing strategies include tool result clearing, which removes earlier tool outputs that are no longer needed, freeing context and keeping the agent focused. There is also thinking-block clearing. These deliberate edits help manage the context window in long agentic runs.

91. An evaluator-optimizer loop keeps running indefinitely because the evaluator is never fully satisfied. What is the best fix that preserves the pattern's value?

A. Run the loop in parallel forever
B. Add a maximum iteration cap plus a 'good enough' acceptance threshold so the loop terminates predictably
C. Remove the evaluator entirely
D. Make the evaluator always approve on the first try

Correct:
Add a maximum iteration cap plus a 'good enough' acceptance threshold so the loop terminates predictably
Explanation:
Evaluator-optimizer loops must terminate, so you bound them with a maximum iteration count and a concrete acceptance threshold ('good enough') rather than chasing perfection. This preserves the quality gains of iterative critique while controlling cost and latency. Unbounded loops are a common reliability failure.

92. An agent occasionally takes irreversible actions (deleting records). Following agent safety guidance, what is the most appropriate architectural control before such an action executes?

A. Let the agent act freely and audit the logs afterward
B. Switch to a smaller model to reduce the chance of action
C. Require a human-in-the-loop approval or an explicit permission check for high-impact, irreversible actions
D. Increase the thinking budget so the model is more careful

Correct:
Require a human-in-the-loop approval or an explicit permission check for high-impact, irreversible actions
Explanation:
For consequential or irreversible actions, Anthropic's guidance is to keep a human in the loop or enforce explicit permission checks before the action runs. Tools like PreToolUse hooks and permission modes in the Agent SDK and Claude Code exist to gate such actions. This bounds blast radius when the agent is wrong.

93. An architect wants to combine cost optimizations on a large asynchronous job with a big stable prompt prefix. Which combination is valid and most economical?

A. There is no way to combine cost optimizations
B. Batch API requires disabling caching
C. Batch API plus prompt caching, since the batch discount stacks with caching for very large savings
D. Caching only works synchronously and never with batch

Correct:
Batch API plus prompt caching, since the batch discount stacks with caching for very large savings
Explanation:
The Batch API's 50% discount stacks with prompt caching, so a cached batch request can cost a small fraction of a standard non-cached request. Combining them is the most economical path for large asynchronous jobs with a stable prefix. Knowing these stack is a practical cost-architecture skill.

94. A team wants a hook to run after a tool call fails so they can log diagnostics and notify on-call. Which lifecycle event is most appropriate?

A. UserPromptSubmit
B. PreToolUse
C. SessionStart
D. PostToolUseFailure

Correct:
PostToolUseFailure
Explanation:
The Agent SDK exposes a Post ToolUseFailure event that fires specifically when a tool call fails, which is the right hook point for failure logging and alerting. Pre ToolUse fires before execution and UserPromptSubmit/SessionStart fire at unrelated moments. Matching the event to the trigger you care about is essential to reliable hook design.

95. A remote MCP server must be reachable by clients over the network and support bearer-token authentication. Which transport supports this?

A. MCP does not support remote servers
B. A local Unix pipe only
C. Stdio transport
D. Streamable HTTP transport, which uses HTTP POST (with optional Server-Sent Events) and supports standard HTTP auth like bearer tokens and API keys

Correct:
Streamable HTTP transport, which uses HTTP POST (with optional Server-Sent Events) and supports standard HTTP auth like bearer tokens and API keys
Explanation:
Streamable HTTP transport uses HTTP POST for client-to-server messages with optional Server-Sent Events for streaming, and it supports standard HTTP authentication including bearer tokens, API keys, and custom headers. This enables secure remote MCP servers. Stdio is local-only and cannot serve remote network clients with HTTP auth.

96. A team wants consistent, deterministic-as-possible formatting for the same input across runs in a structured extraction task. Which setting most directly reduces output variability?

A. Increasing budget_tokens
B. Adding more stop sequences
C. Lowering temperature toward 0
D. Raising temperature toward its maximum

Correct:
Lowering temperature toward 0
Explanation:
Lowering temperature toward 0 reduces sampling randomness, making outputs more consistent and repeatable for extraction tasks. Higher temperature increases creative variability, which is the opposite of what extraction wants. For hard guarantees on structure, pair low temperature with structured outputs.

97. Anthropic warns that frameworks can obscure what is actually happening. What is their recommended approach when deciding whether to adopt an agent framework?

A. Always adopt the most popular framework regardless of need
B. Start by using the LLM API directly since many patterns are a few lines of code, and if you use a framework, ensure you understand the underlying code
C. Never use any framework under any circumstances
D. Use a framework only if it hides all prompts from you

Correct:
Start by using the LLM API directly since many patterns are a few lines of code, and if you use a framework, ensure you understand the underlying code
Explanation:
Anthropic advises starting with direct API usage because many agentic patterns are just a few lines of code, and frameworks can add layers of abstraction that obscure prompts and make debugging harder. If you do adopt a framework, you should understand the underlying code so abstractions do not hide errors. This preserves transparency and debuggability.

98. An organization wants to block a specific MCP server from being used in Claude Code, even if an allowlist would otherwise permit it. How do allow and deny rules interact?

A. Allow rules always win over deny rules
B. The most recently added rule wins regardless of type
C. Allow and deny rules cannot coexist
D. Deny rules take absolute precedence over allow rules

Correct:
Deny rules take absolute precedence over allow rules
Explanation:
In Claude Code permission configuration, the denylist takes absolute precedence over the allowlist. So, a server on the deny list is blocked even if an allow rule would otherwise permit it. This 'deny wins' precedence is a standard safe default for permission systems.

99. A research agent must handle tasks where the number and nature of subtasks cannot be predicted in advance: a central model must dynamically break the work down, delegate pieces, and synthesize the results. Which pattern is the best fit?

A. Routing to one of three static categories
B. Prompt chaining with a fixed sequence of steps
C. Sectioning a single prompt into headers
D. Orchestrator-worker

Correct:
Orchestrator-worker
Explanation:
In the orchestrator-worker pattern, a central LLM dynamically decomposes a task, delegates subtasks to worker LLMs, and synthesizes their results. It is specifically suited to problems where you cannot predict the subtasks ahead of time, unlike fixed chains or static routing. The orchestrator adapts the plan based on what it discovers.

100. An architect wants the orchestrator to remain in control while still giving workers autonomy over how they complete their subtask. What is the correct division of responsibility?

A. Workers synthesize the final answer and the orchestrator does nothing
B. The orchestrator executes every subtask itself and ignores workers
C. The orchestrator owns decomposition, delegation, and final synthesis; each worker owns how it completes its assigned subtask
D. Workers decide the overall goal; the orchestrator just relays text

Correct:
The orchestrator owns decomposition, delegation, and final synthesis; each worker owns how it completes its assigned subtask
Explanation:
In orchestrator-worker, the orchestrator owns the high-level plan: it decomposes the task, delegates subtasks, and synthesizes results into the final output. Workers own the local problem-solving for their assigned subtask. This separation keeps global coherence with the orchestrator while parallelizing focused work.